

Análisis y Diseño con Patrones

Práctica 1: Patrones de Análisis y Arquitectónicos

Una tienda que vende componentes hardware (ratones, pantallas, etc...) por internet nos ha pedido que le diseñemos una parte de un sistema software para gestionar las ventas de estos componentes.

Los clientes de la tienda se identifican con su dni y también proporcionan su nombre. Un subconjunto de clientes son socios de la tienda. Para estos, el sistema registra el número de socio y el año en el que se dieron de alta como socios de la tienda.

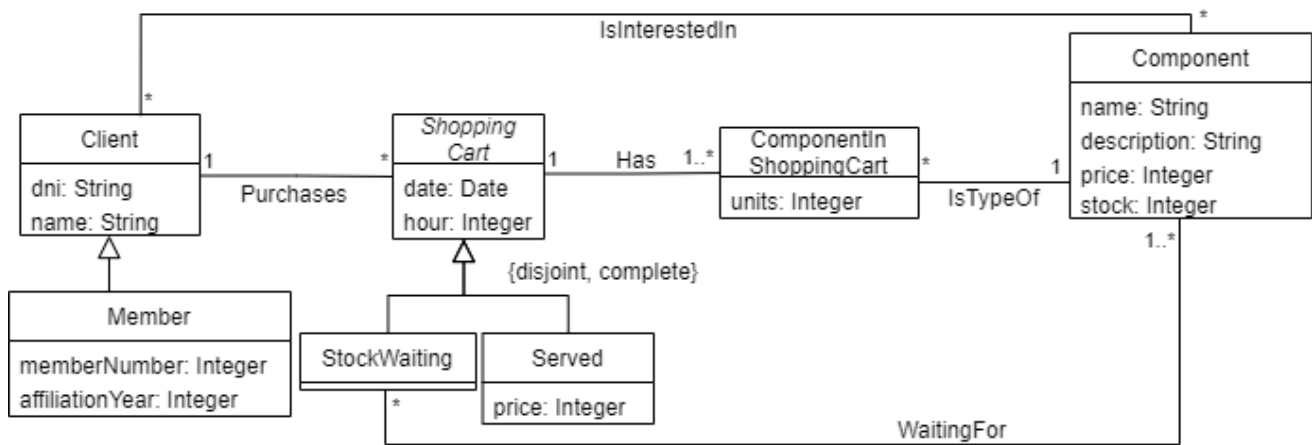
Los componentes que ofrece la tienda se identifican por su nombre y conocemos también su descripción, su precio y el stock. Por ejemplo, el componente con nombre RatonLogitechg34 tiene la descripción del ratón, un precio de 35 euros y un stock de 7 unidades.

Las compras que hacen los clientes se identifican por la fecha y la hora. Las compras pueden estar en espera de stock de alguno de los componentes o servidas. En el caso de las compras en espera de stock, el sistema registra los componentes que se están esperando. Asumimos que si la compra está en espera de un componente, está a la espera de todas las unidades del componente. En el caso de las compras servidas, el sistema almacena su precio.

Los clientes pueden mostrar su interés en componentes para recibir noticias de estos.

Disponemos ya de un análisis previo que podemos utilizar como punto de partida, pero habrá que corregir y mejorar. A continuación disponéis del diagrama estático de análisis por esta parte del software:

[Diagrama estático de análisis](#)



Claves de las clases:

- *Client*: *dni*
- *Member*: *memberNumber*
- *ShoppingCart*: *date* + *hour*
- *ComponentInShoppingCart*: *date* + *hour* (*ShoppingCart*) + *name* (*Component*)
- *Component*: *name*

Restricciones de integridad:

- El stock y precio de un componente es superior a 0.
- El número de unidades de un componente en una compra es superior a 0.
- Los componentes en espera de stock de una compra tienen que ser componentes de aquella compra.
- El precio de una compra servida es la suma de los precios de sus componentes multiplicado por la cantidad de los componentes. Si la compra la hace un socio, el importe se decrementará en un 10%.

Pregunta 1 (15%)

Enunciado

Queremos diseñar la operación *getPrice()*: *Integer* de la clase *ShoppingCart* que calcula el precio de una compra. El precio de una compra es la suma de los precios de los componentes multiplicado por la cantidad de los componentes. Si la compra la hace un socio, este precio se decrementará en un 10%. Se pide:

- a) Propón el pseudocódigo de la operación. Hace falta que proporciones el pseudocódigo de todas las clases y operaciones involucradas en la definición de la operación. La operación tiene que cumplir con el principio de abierto y cerrado, sabiendo que en el futuro otros tipo de clientes serán añadidos y cada tipo de cliente podrá tener descuentos diferentes a los que tienen los socios.
- b) Revisa el código anterior y explica claramente, con un ejemplo, como habéis garantizado el bajo acoplamiento y la alta cohesión.

Solución

- a) A continuación disponéis del pseudocódigo de la operación *getPrice(): Integer* de la clase *ShoppingCart* que calcula el precio de una compra. El principio de abierto y cerrado se garantiza con la operación polimórfica *getDiscount(): Integer* que se ha definido en *Cliente* y que permite que si añadimos nuevos tipos de clientes sólo habrá que añadir las nuevas clases (correspondientes a los nuevos tipos de clientes) e implementar la operación *getDiscount(): Integer*.

```
public class ShoppingCart
{
    private Date date;
    private Integer hour;
    private Client client;
    private Array<ComponentInShoppingCart> componentsInSC;

    public Integer getPrice()
    {
        Integer price = 0;

        for each (ComponentInShoppingCart csc in componentsInSC) {
            price = price + csc.getPrice();
        }

        price = price - (price * client.getDiscount() / 100);

        return price;
    }
}
```

```
public class ComponentInShoppingCart
```

```
{
    private Integer units;
    private Component comp;

    public Integer getPrice()
    {
        price = units * comp.getPrice();
        return price;
    }
}

public class Component
{
    private String name;
    private String description;
    private Integer price;
    private Integer stock;

    public Integer getPrice()
    {
        return price;
    }
}

public class Client
{
    private String dni;
    private String name;
    private static final Integer CLIENTDISCOUNT = 0;

    public Integer getDiscount()
    {
        return CLIENTDISCOUNT;
    }
}

public class Member extends Client
```

```
{  
    private Integer memberNumber;  
    private Integer affiliationYear;  
    private static final Integer MEMBERDISCOUNT = 10;  
  
    public Integer getDiscount()  
    {  
        return MEMBERDISCOUNT;  
    }  
}
```

- b) Bajo Acoplamiento: Este principio se satisface en la operación ya que invocamos desde las operaciones de las clases solo operaciones sobre objetos que las clases ven (por ejemplo, si estos objetos son un atributo de la clase). Si nos fijamos en la operación *getPrice()*: *Integer* de la clase *ShoppingCart*, esta operación solo invoca operaciones sobre los objetos cliente y componentes in shopping cart que los conoce puesto que son atributos de la clase. Esto hace que el acoplamiento de la clase *ShoppingCart* se mantenga lo más bajo posible.

Alta cohesión: Este principio se satisface puesto que todas las clases solo tienen una única operación y los atributos y operaciones de cada clase están muy relacionados. Por ejemplo, la clase *Componente* es muy cohesiva puesto que los atributos que hay definidos en la clase y su operación para obtener el precio de un componente están muy relacionados.

Pregunta 2 (30%)

Enunciado

La tienda para la cual estamos diseñando este sistema software se ha dado cuenta que solo está registrando los intereses actuales de sus clientes en los componentes que vende. Para hacer acciones de marketing e incrementar las ventas de sus componentes, la tienda quiere registrar los intereses que tenían sus clientes en el pasado. Notáis que durante el tiempo, un cliente puede estar interesado en un componente, dejar de estarlo y volver a estar interesado.

Nos proponen buscar una solución que nos permita representar esta información en nuestro sistema. En concreto, se pide:

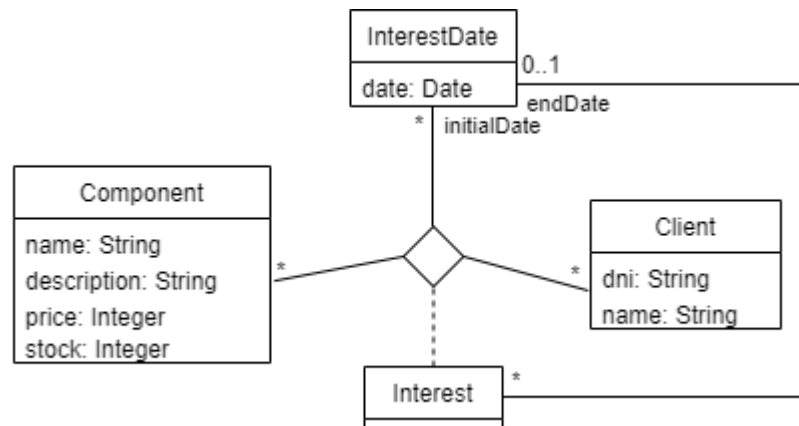
- a) Aplica uno de los patrones de análisis que hay explicados a los materiales para representar esta información, explica cual y razona su conveniencia.
- b) Propón una solución detallada en forma de diagrama estático de análisis (incluyendo los nuevos atributos que aparezcan, las restricciones de integridad y las multiplicidades de las asociaciones). Muestra solo las clases que cambien respecto al diagrama del enunciado.
- c) Propón el pseudocódigo de la operación *interestedComponentsPerDate(date: Date): Set(String)* de la clase *Client* que devuelve los nombres de los componentes en los que estaba interesado el cliente en la fecha indicada en el parámetro. Podéis asumir que el parámetro es una fecha del pasado. Si lo necesitáis, podéis suponer la existencia de la operación *insidePeriod(initialDate: Date, endDate: Date, date: Date): Boolean* que devuelve cierto, si la fecha *date* está dentro del periodo [*initialDate*, *endDate*] y falso, en caso contrario. Esta operación la podéis invocar desde donde la necesitáis y no hace falta que deis el pseudocódigo.

Solución

- a) Según el diagrama estático de análisis del enunciado, solo se registra el interés que actualmente tienen los clientes en algunos componentes. Para poder registrar los intereses que tenían sus clientes en el pasado podemos utilizar el patrón Asociación Histórica.

Este patrón nos permite transformar la asociación binaria *IsInterestedIn* en una asociación ternaria que permite registrar el histórico de los intereses de los clientes en los componentes.

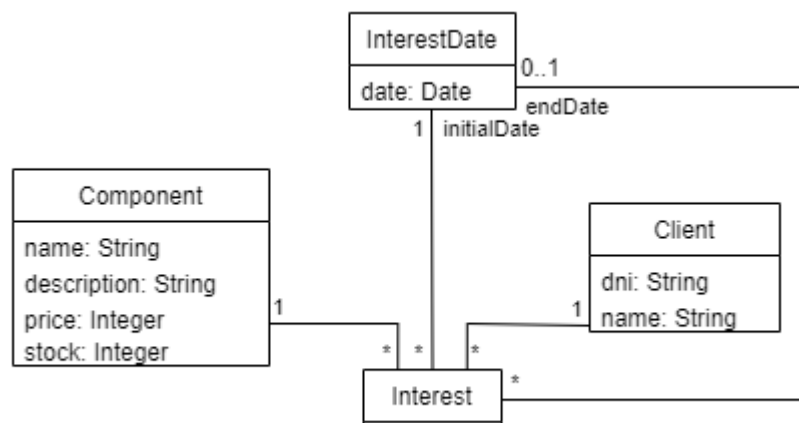
- b) A continuación disponéis de la parte del diagrama estático de análisis que se ve modificado después de la aplicación del patrón Asociación Histórica.



Restricciones de integridad:

- La fecha inicial del interés es anterior a la fecha de finalización del interés.
- Los intereses de un cliente por un mismo componente no pueden tener intervalos solapados.

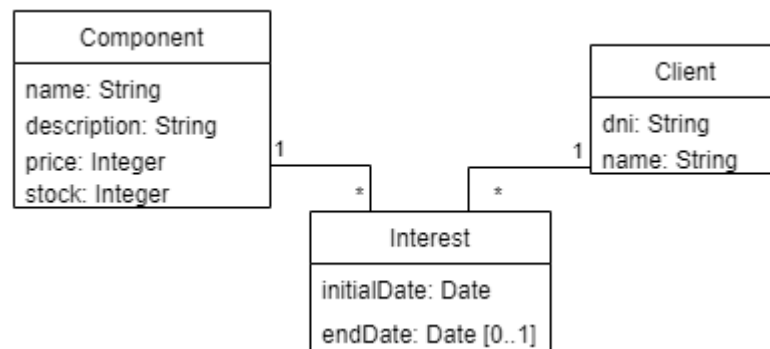
Los lenguajes de programación orientados a objetos actuales no permiten definir asociaciones ternarias, ni clases asociativas. Para poder trabajar con estos constructores, hay que aplicar una pequeña transformación al diagrama estático de análisis para obtener otro diagrama de clases que represente la misma semántica pero con constructores que los lenguajes de programación orientados a objetos permite representar. Aplicando esta transformación obtendremos:



Restricciones de integridad (adicionales en esta solución):

- Clave de Interest: *dni (Client) + name (Component) + initialDate (Date)*
- Los intereses de un cliente por un componente no se pueden solapar.

De hecho, como *InterestDate* se podría considerar como un tipo de datos, y los tipos de datos se pueden poner normalmente como atributos, esta solución la podemos transformar en:



Este diagrama es el que utilizaremos para hacer el apartado siguiente.

c)

```

public class Client
{
    private String dni;
    private String name;
    private Array<Interest> interest;

    public Array<String> interestedComponentsPerDate(date: Date)
    {
        Array<String> interestedComponents;
        Integer index = 0;
        for each (Interest i in interest) {
            if (insidePeriod(i.getInitialDate(), i.getEndDate(), date)) {
                name = i.getComponentName();
                interestedComponents [i] = name;
                i++;
            }
        }
        return (interestedComponents)
    }
}
  
```



```
public class Interest
{
    private date initialDate;
    private date endDate;
    private Component comp;

    public date getInitialDate()
    {
        return initialDate;
    }

    public date getEndDate()
    {
        return endDate;
    }

    public String getComponentName()
    {
        return comp.getName();
    }
}

public class Component
{
    private String name;
    private String description;
    private Integer price;
    private Integer stock;

    public String getName()
    {
        return name;
    }
}
```

Pregunta 3 (15%)

Enunciado

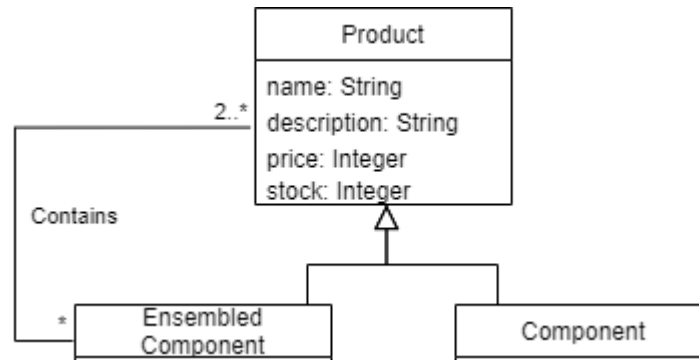
La tienda de componentes decide expandir su negocio vendiendo, además de los componentes básicos (ratones, pantallas, etc...) que vendía hasta ahora, componentes ensamblados de componentes básicos y/u otros productos ensamblados. Por ejemplo, una CPU es un componente ensamblado de dos componentes básicos una ALU y una unidad de control. Un ordenador es un componente ensamblado de varios componentes básicos, como por ejemplo, un ratón y otros componentes ensamblados, como por ejemplo una CPU. Para simplificar, asumimos que en estos nuevos componentes ensamblados solo hay **una** unidad de cada uno de los componentes y/o componentes ensamblados que forman parte. Los componentes ensamblados, igual que los componentes básicos que se vendían hasta el momento, se identifican por su nombre, tienen una descripción, un precio y un stock. En un producto ensamblado no puede haber ni de forma directa ni de forma indirecta el mismo producto ensamblado.

Nos proponen buscar una solución que nos permita representar estos nuevos componentes ensamblados en nuestro sistema (podéis partir del diagrama estático de análisis del enunciado). En concreto, se pide:

- a) Aplica uno de los patrones de análisis que hay explicados a los materiales para representar esta información, explica cual y razona su conveniencia.
- b) Propón una solución detallada en forma de diagrama estático de análisis (incluyendo los nuevos atributos que aparezcan y las multiplicidades de las asociaciones). Muestra solo las clases que cambien respecto al diagrama del enunciado.

Solución

- a) Para representar esta información aplicamos el patrón Objeto Compuesto. Los productos que ofrecerá la tienda serán de dos tipos: Componentes (los que ofrecía hasta ahora) y Componentes Ensamblados (agrupación de productos). Estos últimos contienen como mínimo 2 productos (que pueden ser o bien componentes o bien otros componentes ensamblados).
- b) A continuación disponéis del fragmento del diagrama estático de análisis que representa la información de los diferentes tipos de producto. Se han añadido también las restricciones de integridad.



Restricciones de integridad:

- Un componente ensamblado no puede contener ni de forma directa ni de forma indirecta el mismo producto ensamblado.

Pregunta 4 (10%)

Enunciado

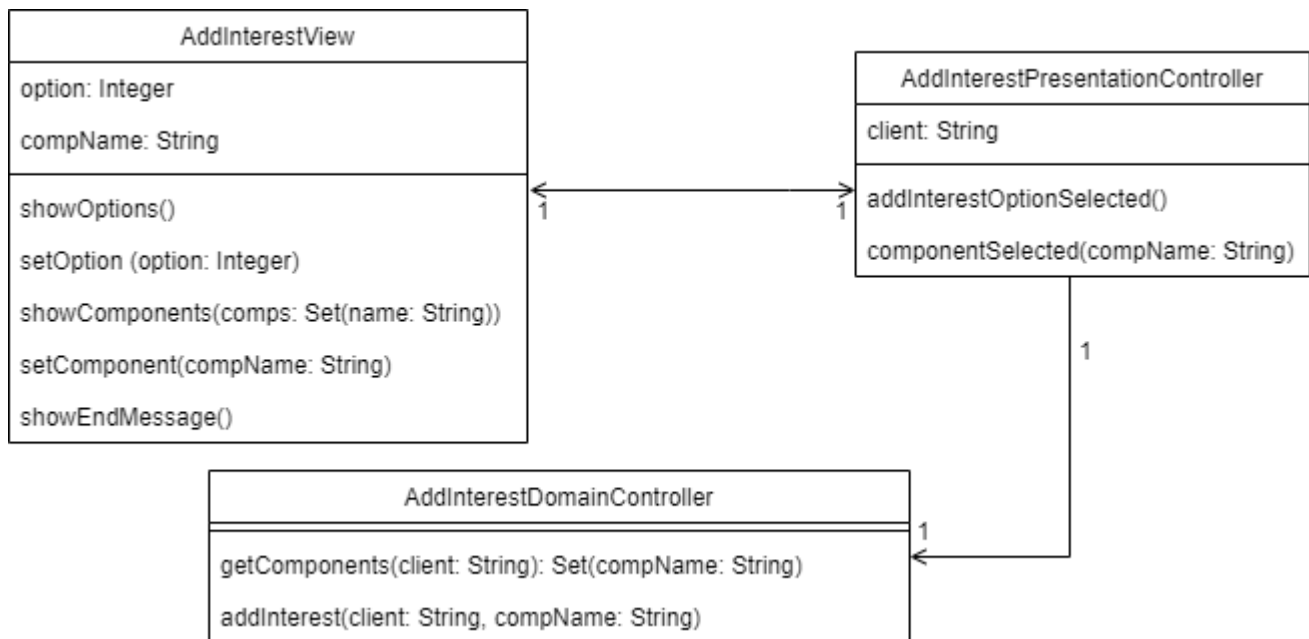
Queremos definir la capa de presentación de nuestro sistema y aplicar el patrón de arquitectura Modelo, Vista y Controlador (MVC) para el caso de uso *Añadir un nuevo interés para un componente*. Este caso de uso permite a un cliente de la tienda, desde su área privada, seleccionar la funcionalidad/opción para añadir un nuevo interés. En respuesta, el sistema le mostrará todos los componentes en los que no está interesado actualmente el cliente. El cliente seleccionará el componente sobre el que quiere registrar su interés, el sistema registrará este interés y le mostrará el mensaje "Interest successfully registered". Para hacer este ejercicio tenéis que partir del diagrama estático del enunciado.

Se pide:

- Diagrama de clases del MVC para este caso de uso incluyendo las operaciones que sean necesarias (las operaciones tienen que contener los parámetros que necesiten y los tipos de retorno, si tienen). El diagrama de clases y las operaciones tienen que ser específicas para este caso de uso.
- Describe brevemente qué hace cada operación que aparece en el diagrama de clases anterior.

Solución

- a) Aplicaremos el MVC a la capa de Presentación. La clase `AddInterestView` representa la vista del patrón, la clase `AddInterestPresentationController` representa el controlador y la clase `AddInterestDomainController` representa el controlador de la capa de dominio. El diagrama de clases del MVC correspondiendo al caso de uso sería:



- b) A continuación se describen todas las operaciones de las clases definidas en el diagrama de clases del apartado a). Asumimos que la clase `AddInterestPresentationController` dispone del identificador del cliente que está utilizando el caso de uso (se ha identificado el cliente cuando ha iniciado su sesión).

Operaciones de la clase `AddInterestView`:

- `AddInterestView::showOptions()` muestra las diferentes opciones correspondientes a los diferentes casos de uso (entre ellos el que permite añadir un interés en un componente) a la pantalla.
- `AddInterestView::setOption(option)` registra el caso de uso seleccionado y lo notifica al controlador. Esta operación es opcional, solo si se quiere que la vista grabe el caso de uso seleccionado.
- `AddInterestView::showComponents(comps: Siete (name:String))` muestra a la pantalla un desplegable con todos los nombres de los componentes que hay al conjunto `comps`.
- `AddInterestView::setComponent(compName: String)` registra el componente en el que se quiere registrar el interés.

- `AddInterestView::showEndMessage()` muestra a la pantalla el mensaje “Interest successfully registered”.

Operaciones de la clase `AddInterestPresentationController`:

- `AddInterestPresentationController::addInterestOptionSelected()` detecta que el cliente ha escogido la opción para añadir un nuevo interés e invoca a la operación para añadir este interés.
- `AddInterestPresentationController::componentSelected(*compName: String)` detecta que el cliente ha seleccionado el componente en el que se quiere mostrar el interés e invoca a la operación que graba el interés.
-

Operaciones de la clase `AddInterestDomainController`:

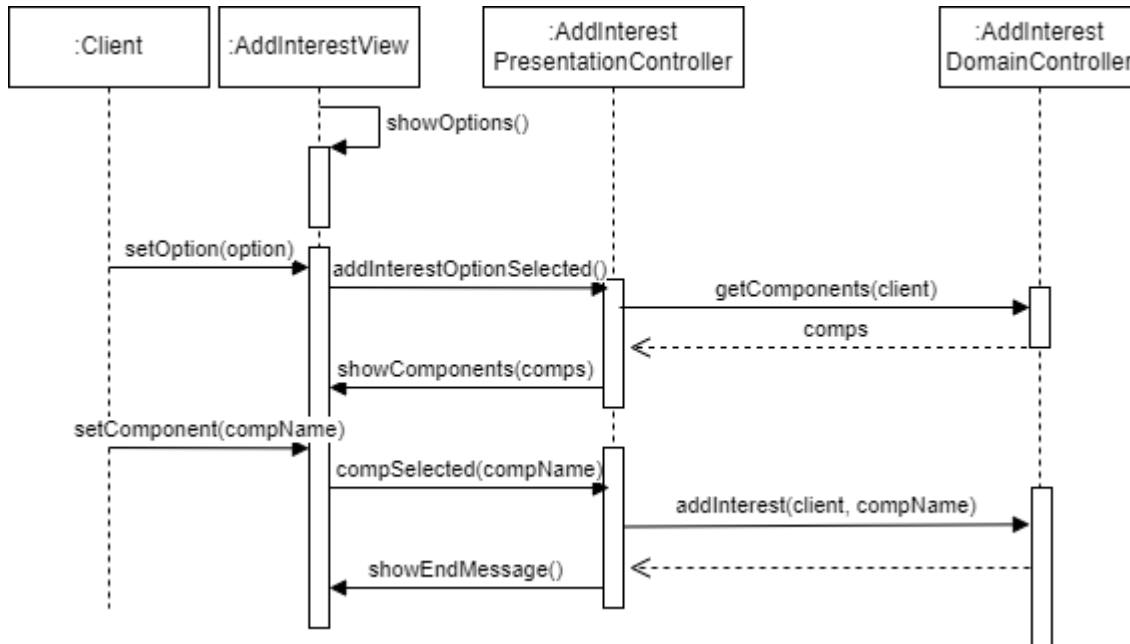
- `AddInterestDomainController::getComponents(cliente:String): Siete(compName: String)` obtiene los componentes en los que el cliente no está actualmente interesado.
- `AddInterestDomainController::addInterest (cliente:String, compName:String)` graba el interés del cliente en el componente.

Pregunta 5 (10%)

Enunciado

Propón el diagrama de secuencia o pseudocódigo del caso de uso *Añadir un nuevo interés para un componente* (del ejercicio anterior), desde que se muestra la pantalla para que el cliente pueda seleccionar del menú la funcionalidad/opción de añadir un nuevo interés para un componente hasta que el sistema le muestra el mensaje “Interest successfully registered”.

Solución



Pregunta 6 (10%)

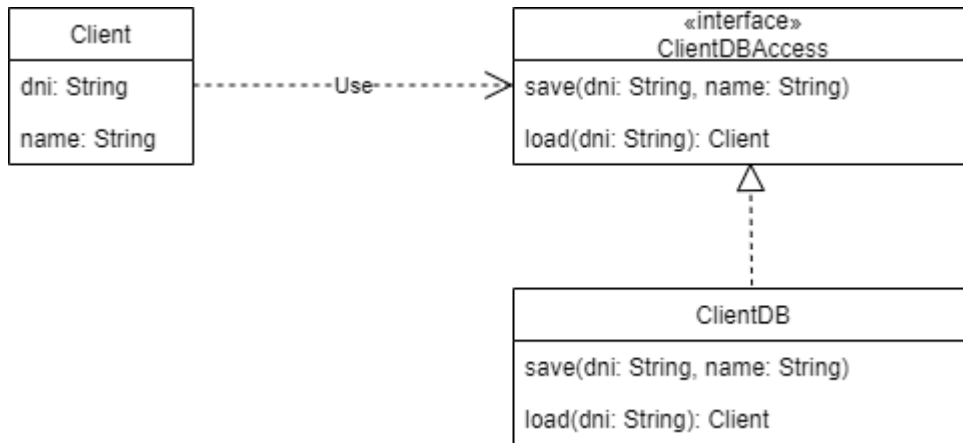
Enunciado

Seguimos diseñando la arquitectura del sistema y hemos decidido utilizar una arquitectura con 3 capas: presentación, dominio y servicios técnicos (en nuestro caso, capa de de datos). Se pide:

- Haced un diseño (con un diagrama de clases) donde se muestre cómo se podrían comunicar las clases que ya tenemos definidas (mostradas en el diagrama estático de este enunciado) y que pertenecen a la capa de dominio con la base de datos. Esta comunicación se tiene que hacer mediante la capa de datos y tiene que permitir obtener y guardar la información de las clases definidas a la capa de dominio. Hacedlo solo para una clase, por ejemplo, la clase *Client*, el resto de clases se podrían tratar de forma similar. La solución que propongáis tiene que satisfacer el principio de inversión de dependencias.
- Explicad claramente cómo habéis garantizado, en el apartado anterior, el principio de inversión de dependencias.

Solución

- a) El diagrama de clases que muestra la comunicación entre la clase Cliente (de la capa de dominio) y la base de datos es el siguiente:



- b) El principio de diseño de Inversión de Dependencias se garantiza en el diagrama de clases anterior puesto que la clase Client no depende de la clase ClientDB sino que depende de una interface ClientDBAccess (abstracción). Los detalles, es decir, el acceso a la DB están en la clase ClientDB que solo depende de la interface ClientDBAccess (abstracción).

Pregunta 7 (10%)

Enunciado

Para acabar de diseñar la arquitectura en 3 capas del sistema, usando las tres capas clásicas de muchos sistemas de información:

1. Presentación
2. Dominio
3. Servicios técnicos (Datos)

Queremos reflexionar sobre a qué capa pertenece cada una de las clases que han ido surgiendo en el enunciado (y a la solución que has elaborado hasta ahora):

- Clases del diagrama de clases del ejercicio 2.
- Clases del diagrama de clases del ejercicio 4.
- Clases del diagrama de clases del ejercicio 6.

- a) Indica qué clases (e interfaces, si hay) pertenecen a la capa de dominio. Razona tu respuesta.
- b) Indica qué clases (e interfaces, si hay) pertenecen a la capa de presentación. Razona tu respuesta.
- c) Indica qué clases (e interfaces, si hay) pertenecen a la capa de servicios técnicos (en nuestro caso, capa de datos). Razona tu respuesta.

Solución

- a) Las clases de la capa de dominio es donde hay la lógica de negocio, donde trabajamos el problema que nuestro sistema resuelve.

Esto incluye, naturalmente, las clases las instancias de las cuales representan objetos, documentos o valores del mundo real, como un componente, un socio, etc. Por lo tanto, todas las clases del diagrama de clases del ejercicio 2 pertenecen a la capa de dominio.

También incluye la clase que ofrece operaciones del nivel del dominio para ser usadas por la capa de presentación, el que denominamos controladores de dominio. En nuestro caso, AddInterestDomainController es un controlador de la capa de dominio que estaba definido en el diagrama de clases del ejercicio 4.

Finalmente, la interface ClientDBAccess (del diagrama de clases del ejercicio 6), que se ha añadido para satisfacer el principio de inversión de dependencias, también está definida a la capa de dominio.
- b) Las clases de la capa de presentación son aquellas que gestionan la presentación de la información (vistas del MVC) y la recepción de los acontecimientos al sistema (controladores de presentación del MVC). En nuestro caso estas clases serían AddInterestView y AddInterestPresentationController.
- c) Las clases de la capa de datos son aquellas que gestionan el acceso a la DB. En nuestro caso estas clases serían ClientDB y el resto de clases correspondientes a las clases de la capa de dominio para acceder a la DB.